

CS3160 Lab 1

RISC-V Assembly: Toolchain, ELF and Assembly Programs

What you will do today

You will write RISC-V assembly programs, assemble them into executables, run them on a RISC-V simulator, and check them with the same automated tests.

By the end of the session you should be able to answer: what does the assembler actually produce, where does my code and data end up in memory, and how do I see what the processor did instruction by instruction? Those three questions are the foundation for the rest of the course, where you will build a processor simulator of your own.

Your folder is collected at the end of the session, and **only what is in it at that moment is graded**. You can keep working on problems afterwards for your own practice.

1 The tools

Two programs that are essential for this lab are already installed on the lab machines.

- **riscv-none-elf-gcc** — the compiler and assembler. It turns your `.s` source into a RISC-V executable. Note the prefix: plain `gcc` on these machines builds programs for the lab machine's own processor, which is *not* RISC-V.
- **spike** — a RISC-V simulator. The lab machines cannot execute RISC-V instructions directly, so Spike pretends to be a RISC-V processor and runs your program one instruction at a time.

Run the following commands and check if you are getting the outputs shown:

```
$ source /opt/cs3160/setup.sh
$ riscv-none-elf-gcc --version
riscv-none-elf-gcc (xPack GNU RISC-V Embedded GCC ...) 13.1.0
$ spike --version
Spike RISC-V ISA Simulator 1.1.1-dev
```

2 The folder you were given

```
lab01/
  handout.pdf          this document
  details.txt          YOU MUST FILL THIS IN
  docs/               reference material
  programs/
```

asms/	your assembly programs go here
custom/	start-up code and linker script (do not edit)
Makefile	builds and runs your programs
tests/	the automated tests
Makefile	make, make test, make clean

Three folders appear once you build: `programs/bins/` (the executables), `programs/dumps/` (human-readable disassembly), and `programs/runs/` (what happened when the program ran).

File name conventions used all semester

<code>.s</code>	RISC-V assembly source. Lower case <code>.s</code> , never <code>.S</code> .
<code>.r5o</code>	A RISC-V executable (an ELF file).
<code>.r5ob</code>	The same program as a flat binary image.
<code>.dump</code>	Disassembly, produced by <code>objdump</code> .
<code>.iss</code>	A record of a Spike run.

3 Building and running

From inside `lab01/`:

```
$ make           # assemble everything in programs/asms/
$ make test      # run the automated tests against your programs
$ make clean     # delete everything that was generated
```

From inside `lab01/programs/` you get finer control:

```
$ make           # build all
$ make bins/1-even.r5o  # build just one program
$ make run       # run every program on Spike
$ make runs/1-even.iss  # build and run just one
```

4 Anatomy of a program

Open `programs/asms/data_vars.s`. It is a worked example: read it, build it, run it. Every program you write has the same skeleton.

```
.section .data           # variables live here
    .align 2             # start on a 4-byte boundary
myvar1: .word 0xc001      # a 32-bit variable, named myvar1

.section .text           # instructions live here
    .globl main           # make 'main' visible to the start-up code
main:
    la    x5, myvar1      # x5 = the ADDRESS of myvar1
    lw    x11, 0(x5)      # x11 = the CONTENTS of myvar1
    li    x10, 0          # exit status 0
    ret                  # return -- this stops the machine
```

Two directives split the program in half. `.section .data` introduces the variables; `.section .text` introduces the instructions. `.globl main` matters because your code is not the first thing that runs.

Why main must return

When Spike starts your program it does not jump to `main`. It jumps to start-up code we supply in `programs/custom/crt.S`, which sets up the registers and the stack pointer and only then calls `main`. When `main` returns, that same start-up code shuts the machine down cleanly.

- *If `main` never returns — if it ends by jumping to itself, which is a common habit — the machine never stops. The simulator runs until it hits an instruction limit, and the tests report that your program did not finish. Always end `main` with `ret`.*

Naming registers

There are 32 registers, `x0` to `x31`. **In this lab, refer to them by number.** You will see other names — `t0`, `a0`, `sp`, `ra` — in the output of Spike and `objdump`. Those are *ABI names*: a convention about which register is used for what when one function calls another. They are aliases, so `x10` and `a0` assemble to identical bits. Lab 2 will introduce the calling convention that gives them meaning.

Use `x5–x7`, `x11–x17` and `x28–x31` freely, and `x10` for the exit status. Leave `x1–x4` alone: the start-up code put things there that your program depends on.

- *`x1` holds the address `ret` returns to. If you use it as scratch, your program will not return — it will jump into your data and try to execute it. The tests report this as “program crashed”.*

Where things are in memory

The linker script `programs/custom/test.ld` fixes the layout. You will rely on these addresses for the rest of the course:

Address	Contents
0x80000000	start-up code from <code>crt.S</code>
0x80001000	<code>tohost</code> how the program says “I am done”
0x80002000	<code>.text</code> your instructions; <code>main</code> starts here
0x80008000	<code>.data</code> your variables

5 Seeing what the processor did

```
$ cd programs
$ make runs/data_vars.iss
$ less runs/data_vars.iss
```

Each executed instruction produces two lines. The first shows the instruction as text; the second shows what it changed.

```
core 0: 0x80002000 (0x00006097) auipc ra, 0x6
core 0: 3 0x80002000 (0x00006097) x1 0x80008000
```

Reading the second line: at address 0x80002000, the instruction encoded as 0x00006097 wrote 0x80008000 into register x1. Loads and stores also name the memory address they touched:

```
core 0: 3 0x80002008 (0x0000a583) x11 0x0000c001 mem 0x80008000
core 0: 3 0x8000200c (0x0062a423) mem 0x80008008 0x0000000b
```

The first is a load: it wrote x11 and read memory at 0x80008000. The second is a store: it wrote 0x0000000b to memory at 0x80008008. This is exactly how the tests check your work — they replay these lines to work out what your program left in memory.

To see your program as the machine sees it, look at the disassembly in `programs/dumps/`. Comparing what you wrote against what the assembler produced is the fastest way to understand pseudo-instructions: `la`, `li` and `ret` are conveniences that expand into one or two real instructions.

6 Reference material

`docs/` contains what you need:

- `docs/RISCV_CARD.pdf` — the instruction and register reference. Keep it open.
- `docs/registers.md` — which registers you may use, a worked example of a loop over an array, and the pseudo-instructions you will need. Read this first if you have not written assembly before.

7 Problems

Write each program in the file named below, which already exists in `programs/asms/` with the variables declared for you.

► *Do not rename, reorder or resize the variables in the `.data` section. The tests replace the input variables with their own values and then read your answer back, so the names and the array sizes must stay exactly as given. An answer hardcoded to match the example values will fail.*

Problem 1 — `1-even.s`

Count how many of the first `n` entries of the array `l` are both non-negative (≥ 0) and even. Store the count in `result`.

With the values supplied ($n = 5$, $l = [2, -1, 7, 5, 3]$) the answer is 1: only 2 qualifies. Note that 0 is both non-negative and even.

Problem 2 — 2-prime.s

Decide whether the value in `a` is prime. Store `1` in `result` if it is, and `-1` if it is not.

Nothing below 2 is prime, so $a = 1$, $a = 0$ and negative values must all give `-1`. With the value supplied ($a = 10$) the answer is `-1`.

Problem 3 — 3-descending.s

Sort the first `n` entries of the array `a` into descending order, largest first. Sort *in place*: the answer is the array itself. Entries beyond the first `n` must be left untouched.

Any algorithm is fine. Think about $n = 0$, $n = 1$, repeated values and negative values — the tests cover all of them.

Problem 4 — 4-histogram.s

Each of the first `n` entries of `marks` is a mark from 0 to 10. Count how many students got each mark: `count[m]` must end up holding the number of marks equal to m , for every m from 0 to 10. `count` starts as all zeros.

You do not need eleven separate comparisons. `count[m]` lives at address $(\text{address of } \text{count}) + 4m$, because each entry is a four-byte word — so the mark itself can be used to compute where to add one.

8 Checking your work

```
$ make test                # all problems
$ python3 tests/run_tests.py 1-even    # just one
```

Each problem is checked against several sets of input values, and you are told which ones your program gets right:

```
1-even Count non-negative even numbers
[pass] case 0: n=5  l=[2, -1, 7, 5, 3]
[FAIL] case 3: n=4  l=[0, -6, 5, 8]
        result: got 1, expected 2
```

This is the same checker used for grading.

9 Submitting

1. Fill in `details.txt` with your name, roll number, and the number of the machine you are sitting at. Your submission is identified by this file.
2. Run `make clean`.
3. Leave everything else where it is. The lab administrator will collect the `lab01` folder at the end of the session.